Exploring the Impacts of Multiple Kernel Sizes of Gaussian Filters Combined to Approximate Computing in Canny Edge Detection

Marcio Monteiro[1], Ismael Seidel[1], Mateus Grellert[1], José Luis Güntzel[1], Leonardo Soares[2], Cristina Meinhardt[1]
[1]*Departamento de Informática e Estatística* - PPGCC - Universidade Federal de Santa Catarina (UFSC)
[2]Instituto Federal do Rio Grande do Sul (IFRS)
{marcio, ismaelseidel}@inf.ufsc.br, leonardo.soares@riogrande.ifrs.edu.br, {mateus.grellert, j.guntzel, cristina.meinhardt}@ufsc.br

2022 IEEE 13th Latin America Symposium on Circuits and System (LASCAS) | 978-1-6654-2008-2/22/$31.00 ©2022 IEEE | DOI: 10.1109/LASCAS53948.2022.9789080

***Abstract*—Image processing applications are currently available on mobile devices, stressing the energy efficiency demands during the hardware design. In these applications, image edge detectors use filters as a preprocessing step to reduce undesirable artifacts and smooth the image. This work explores the combination of Multiplierless Multiple Constant Multiplication and Approximate Computing techniques on the Gaussian filter, investigating three different kernel size impacts on image processing. The impact of approximation is evaluated at different levels using the copy strategy technique on the LSBs of adders. The results show power and area reductions for the kernel sizes under evaluation. For instance, the approximate kernel 7×7 kernel achieved reductions of up to 40% and 48% for the area and power consumption, respectively, compared to the exact version. It shows ample space for design exploration targeting different trade-offs of quality and power results.**

***Index Terms*—Hardware Architecture. Gaussian Filter. Canny edge. Approximate Computing. Energy Efficiency.**

I. Introduction

Identifying objects or regions in an image is an important and costly task in computer vision applications. Edge detectors algorithms are used to extract features in many areas such as medicine, civil engineering, and computer vision [1]. Most of those applications are employed in power-constrained scenarios demanding power-efficient designs. Also, the main components are arithmetic operators like adders, multipliers, and dividers. Those operators are promising candidates for power optimization with Approximate Computing (AxC) techniques [1], especially the adders.

The Canny Algorithm is one of the most used edge detectors and is considered a standard due to the results' reliability with noisy images [2], [3]. Also, this algorithm has multiple steps and generally uses the Gaussian filter to reduce image artifacts. The Gaussian filter is a compute-intensive task that demands energy-efficient architectures. Because of that, Gaussian filters are a frequent case study to evaluate the impact of adopting AxC in applications [4]–[9]. Some common approaches of AxC to optimize the area and power of adder designs are the clustering into accurate and inaccurate parts of the approximate adders [5], hardware reuse [6], approximation of Full Adder (FA) logic [7], [8], and bit-quantization [9]. The development of multi-bit adders exploring AxC at the transistor level significantly impacts the power consumption in FA cells [7]. Also, some techniques explore the carry chain reduction and the generation of partial results with sub-adders [8]. Independently of the AxC approach, one requirement is evaluating the precision reduction.

This work extends a previous evaluation of kernel 5×5 Gaussian filter architecture design presented on [10], which combines logic refactoring and AxC to optimize area and power. We use the V2 architecture from [10] because it has the best trade-off between power and quality. We also evaluate the impact of multiple kernel sizes of the Gaussian filter, including two other kernel sizes architectures: the 3×3 and the 7×7, observing the quality impact on a Canny Edge Algorithm as a real-world application. Similar to [10], this work also combines the copy strategy and Multiplierless Constant Multiplication (MCM) technique to propose different levels of approximation in the evaluated designs, providing area, power, and quality data to help designers on decision-making process according to the design specification.

II. Gaussian Filter Kernels

The Gaussian filter is a Finite Impulse Response (FIR) filter based on a Gaussian function that convolves the input signal. It is a preprocessing step used to smooth and reduce the noise in images on multimedia applications like edge detectors. The main component of the Gaussian filter is a matrix of coefficients, called kernel, obtained by Eq. 1 using a variance (σ^2) and image block size (x,y). Different σ^2 values generate different kernels, but the summation of the coefficients should be equal to 1 [11]. One image block is a sub-matrix of the partitioning of the image, and it is a common approach to alleviate the workload in multimedia applications. The Gaussian filter is a multiplication of matrices and both, kernel and block should have the same size. The Gaussian filter output image is constructed using the convoluted pixels.

$$G(x,y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}} \quad (1)$$

The three most frequent kernel sizes in the literature are 3×3, 5×5, and 7×7. To obtain the coefficients used in this

This study was financed in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) - Finance Code 001 and by Brazilian Council for Scientific and Technological Development (CNPq).

work was used Eq. 1 with σ^2=1.36 and the coordinates of each block. Eq. 2, 3, and 4 presents the coefficients to kernels 3×3, 5×5, and 7×7, respectively. Also, the number of different coefficient values in each kernel is small; kernel 3×3 has three, kernel 5×5 has six, and kernel 7×7 has eight.

$$\begin{bmatrix} 15 & 19 & 15 \\ 19 & 24 & 19 \\ 15 & 19 & 15 \end{bmatrix} \quad (2)$$

$$\begin{bmatrix} 2 & 4 & 5 & 4 & 2 \\ 4 & 9 & 12 & 9 & 4 \\ 5 & 12 & 15 & 12 & 5 \\ 4 & 9 & 12 & 9 & 4 \\ 2 & 4 & 5 & 4 & 2 \end{bmatrix} \quad (3)$$

$$\begin{bmatrix} 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 1 & 2 & 4 & 5 & 4 & 2 & 1 \\ 1 & 4 & 8 & 10 & 8 & 4 & 1 \\ 1 & 5 & 10 & 13 & 10 & 5 & 1 \\ 1 & 4 & 8 & 10 & 8 & 4 & 1 \\ 1 & 2 & 4 & 5 & 4 & 2 & 1 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 \end{bmatrix} \quad (4)$$

The MCM technique [12] was applied to optimize each filter function by decomposing the coefficient into the power of 2 values, e.g., 9δ is $(2^3+2^0)\delta$, allowing the substitution of general-purpose multipliers by sums and shifts. Eq. 5, 6, and 7 are the kernel equations after applying the MCM to kernels 3×3, 5×5, and 7×7, respectively. The variables i_γ are pixels from the input images, and γ is the pixel's position in an image block. Also, the number of operations performed by the filters was reduced, exploring the calculations redundancy in each equation. The kernel size is proportional to the number of arithmetic operations, and, larger sizes increase the room to optimize the implementations by exploring the redundancies. Fig. 1 presents the three architectures, and each architecture uses a register barrier (RB) to control the data flow.

The number of adders and the bit width of each adder changes according to the kernel architecture. To an abstract comparative analysis of the three architectures after the MCM,

Kernel 3×3

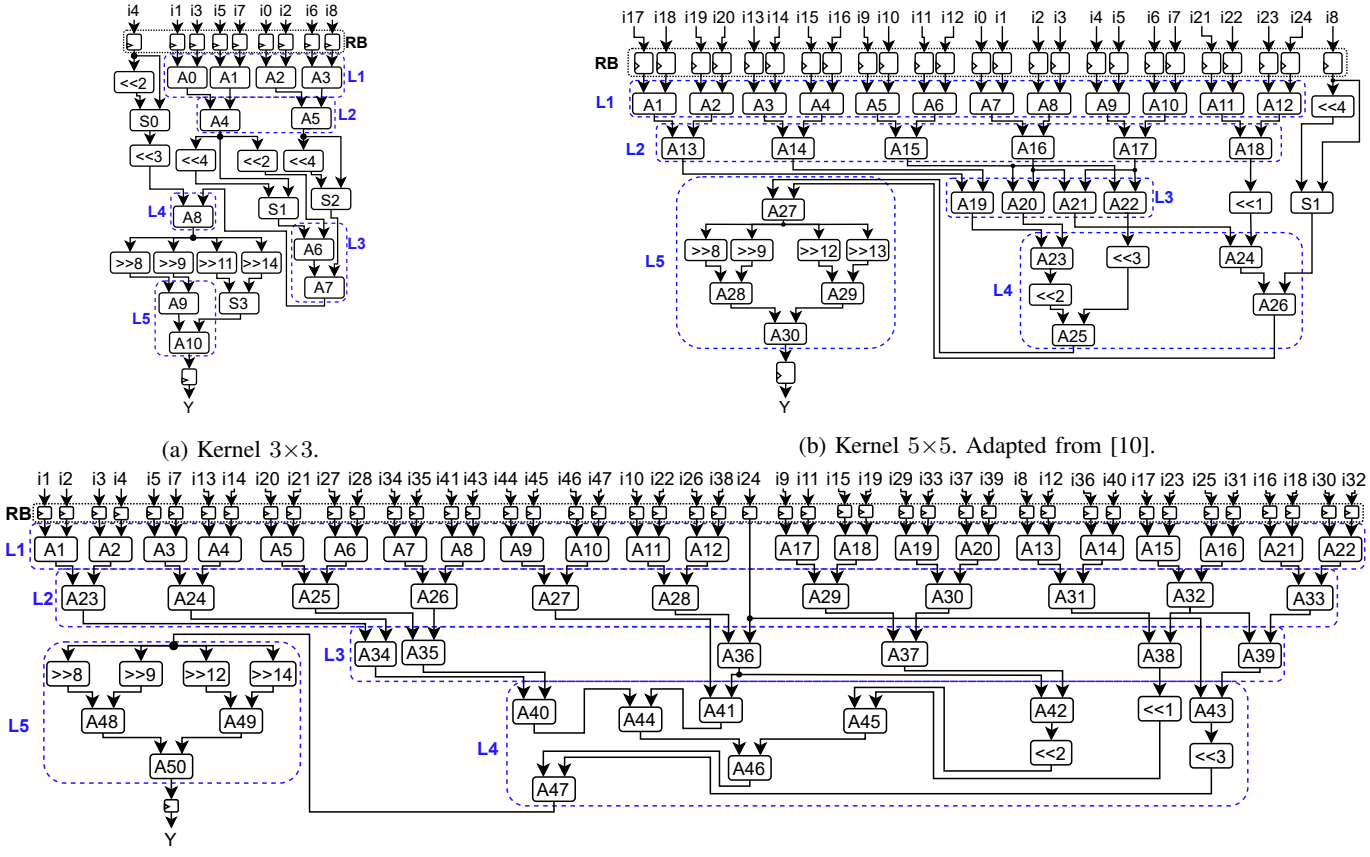
$$Y = 16(i_0 + i_2 + i_6 + i_8 + i_1 + i_3 + i_5 + i_7 + i_4) + 8i_4 + 4(i_1 + i_3 + i_5 + i_7) - (i_0 + i_2 + i_6 + i_8 + i_1 + i_3 + i_5 + i_7) \quad (5)$$

Kernel 5×5

$$Y = 16i_8 + 8(i_4 + i_5 + i_6 + i_7 + i_9 + i_{10} + i_{11} + i_{12}) + 4(i_0 + i_1 + i_2 + i_3 + i_{13} + i_{14} + i_{15} + i_{16} + i_{17} + i_{18} + i_{19} + i_{20} + i_9 + i_{10} + i_{11} + i_{12}) + 2(i_{21} + i_{22} + i_{23} + i_{24}) + (i_0 + i_1 + i_2 + i_3 + i_4 + i_5 + i_6 + i_7) - i_8 \quad (6)$$

Kernel 7×7

$$Y = 8(i_{16} + i_{18} + i_{30} + i_{32} + i_{17} + i_{23} + i_{25} + i_{31} + i_{24}) + 4(i_9 + i_{11} + i_{15} + i_{19} + i_{29} + i_{33} + i_{37} + i_{39} + i_{10} + i_{22} + i_{26} + i_{38} + i_{24}) + 2(i_8 + i_{12} + i_{36} + i_{40} + i_{17} + i_{23} + i_{25} + i_{31}) + (i_1 + i_2 + i_3 + i_4 + i_5 + i_7 + i_{13} + i_{14} + i_{20} + i_{21} + i_{27} + i_{28} + i_{34} + i_{35} + i_{41} + i_{43} + i_{44} + i_{45} + i_{46} + i_{47} + i_{10} + i_{22} + i_{26} + i_{38} + i_{24}) \quad (7)$$



(a) Kernel 3×3.

(b) Kernel 5×5. Adapted from [10].

(c) Kernel 7×7.

Fig. 1: Gaussian filter design of the three kernel sizes evaluated in this work. The RB is the register barrier, the *A*'s are adders, *S*'s are sub-tractors, *Y* is the filter's output, and >>, << are shifters.

we can observe the number of *n*-bit adders and the equivalent of 1-bit FAs. The kernel 3×3 has ten *n*-bit adders, corresponding to 112 1-bit FAs, while the kernel 5×5 has 30 *n*-bit adders equivalent to 268 1-bit FAs, and the kernel 7×7 has 50 *n*-bit adders equal to 451 1-bit FAs.

Also, this work evaluates different levels of approximation applied to the three architectures. The approximate cases use the copy strategy [7] on the Least Significant Bits (LSBs) of the *n*-bit FA only considering the SUM function. The implemented SUM function on approximate adders does not use the carry propagation, so it was removed, increasing power and area savings. Half of the LSBs in each adder were approximated as it introduces less than 2% of Normalized Mean Error Distance (NMED) for 8-bit adders [10]. The three architectures were logically divided into five levels (L1 to L5). The adders included in each level are highlighted in blue dashed lines in Fig. 1. As presented in Table I, each approximate architecture was considered a study case and is identified according to the approximated levels. It results in 32 different cases for each kernel evaluated and the 96 cases were implemented in a C++ framework to be evaluated as a preprocessing step to the OpenCV's Canny Algorithm. It is divided into steps where the input is a gray-scale image, and the output is a binary image with the detected edges. This work explores the three kernels developed into the preprocessing step using all the cases, from the exact to C17 where the copy strategy is applied in all adders. In the experimental evaluation, the Canny Algorithm parameters T1 and T2 are 50 and 150, respectively, and were used to all 96 cases.

To evaluate the approximate Gaussian filters quality was used the *PerformanceConformance* (PfCf) metric [13]. It is the minimum value between the *recall* and *precision* functions, as expressed in Eq. 8, where *recall* is the ratio between the number of edge pixels correctly detected by the approximate version and the number of edge pixels that should be correctly detected (Eq. 9), and *precision* (Eq. 10) is the ratio between the number of pixels correctly detected as an edge over all the pixels detected as edges (i.e., *tp* + *fp*) by the approximate version. The terms *tp, fp*, and *fn* are the detected true positives, false positives, and false negatives. They are obtained using a pixel-by-pixel comparison between the exact Canny Edge Algorithm output and one approximate case output.

$$PfCf(\%) = min(recall, precision) \times 100 \quad (8)$$

$$recall = \frac{tp}{tp + fn} \quad (9) \qquad precision = \frac{tp}{tp + fp} \quad (10)$$

We used ten gray-scale images from the *Berkeley Segmentation Dataset and Benchmark* [14] corrupted by additive Gaussian noise with zero mean and standard deviation equal to 0.1 [6]. Each image is divided into 8-bit per pixel block with the same size as the kernel to use as input to the architecture.

III. Experimental Results

Evaluating the PfCf results, we noticed that using approximate adders in level L5 decreases the results to values below 50%. It is observed to all kernels independently of the remaining levels being approximated or not. Fig. 3 shows the PfCf results obtained for the approximated versions for each kernel. To all the cases with L5 exact, the PfCf to kernel 3×3 is above 50%, while the kernels 5×5 and 7×7 it is above 75%. The approximation effects from L1 to L4 are attenuated by the right shifts performed in all three kernels before the L5 following the finds in [15]. Otherwise, approximating L5 decreases substantially the PfCf compared to the other cases evaluated, reaching values lower than 40% on kernel 3×3 and 7×7. However, this high impact on the quality of the L5 approximation can be acceptable depending on the application requirements.

To all cases with L5 exact, kernel 3×3 decreases the PfCf when the approximate levels increase, showing it is more sensible to approximation than the other kernels. In contrast, for the kernels 5×5 and 7×7, the PfCf results have low variations, showing these kernels are less sensible to approximation. However, deciding which kernel size to use should consider other parameters such as area and power.

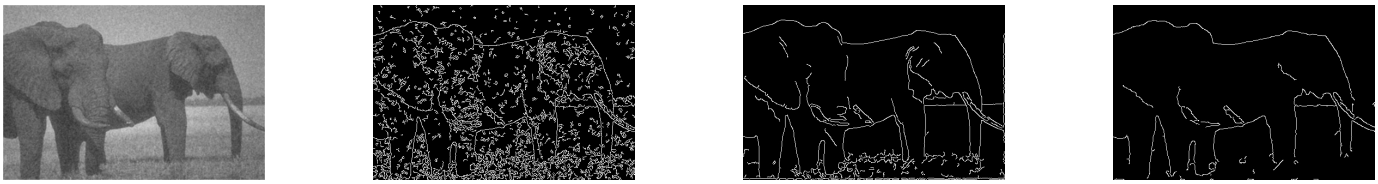
Based on the PfCf results and to evaluate area and power, we implemented the exact architecture and approximate cases from C1 to C17 for each kernel size. In most cases, level 5 is maintained precisely. All those cases were described in Verilog, synthesized, and simulated using Synopsys® tools. The syntheses were performed using the TSMC 45 nm standard cells library, targeting the frequency of 249 MHz to process 4k UHD images at a nominal voltage of 0.9 V. We used the switching activity obtained by processing the benchmark images to perform a more realistic power evaluation.

The total power and area to the three exact architectures were 226 μW with 986 μm^2 to 3×3, to 5×5 is 503 μW with 2276 μm^2, while to 7×7 1097 μW with 3483 μm^2. To all the evaluated cases, on average, the leakage represents 5%, 6%, and 4% of the total power for kernels 3×3, 5×5, and 7×7, respectively. As expected, kernel 3×3 has the lowest power consumption due to the restricted area while kernel 7×7 has the most significant consumption. Based on it, kernel 3×3 is the most suitable architecture to power-constrained applications due to the low area and power consumption for all scenarios under evaluation. However, this kernel shows to be more sensitive to details, and it is illustrated in Fig. 2, with a visual example of the Canny algorithm output using each kernel size. Using the kernel 5×5 as a reference, reducing the kernel size makes the Canny algorithm more sensitive to details while increasing the kernel size reduces the sensibility.

Considering the effects of the approximation applied in each kernel, Table I presents the area and total power reductions related to the correspondent exact architecture of each kernel evaluated. Approximating a single level (cases C1-C5) reduces the area up to 18%, while the total approximation (case 17) reduces the area up to 41% at the cost of high degradation in PfCf. Despite this case, considering L5 exact, the area reduction was between 6% and 40%, with power reduction reaching up to 42%, 49%, and 51% for approximation all levels (case 17) on the kernel 3×3, 5×5, and 7×7, respectively.

TABLE I: Power/Area reduction related to exact kernel architecture to each approximate case.

Kernel size	Power Reduction (%) / Area reduction (%) related to the correspondent exact architecture																
	Case	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16	C17
	AxC Levels	L1	L2	L3	L4	L5	L1,L2	L1,L3	L1,L4	L2,L3	L2,L4	L3,L4	L1,L2, L3	L1,L2, L4	L1,L3, L4	L2,L3, L4	L1,L2, L3,L4	L1,L2,L3, L4,L5
	3×3	13 / 15	2 / -1	7 / 9	3 / 5	5 / 8	18 / 21	19 / 22	15 / 18	9 / 6	5 / 1	8 / 9	25 / 28	21 / 22	21 / 26	12 / 21	28 / 32	33 / 42
	5×5	16 / 15	5 / 1	3 / 4	3 / 10	2 / 8	25 / 29	18 / 34	18 / 21	6 / 23	7 / 27	6 / 10	29 / 44	28 / 35	22 / 41	11 / 31	35 / 42	39 / 49
	7×7	17 / 25	4 / 2	4 / 6	3 / 13	0 / 2	26 / 33	20 / 29	22 / 36	7 / 14	7 / 9	9 / 15	32 / 42	30 / 40	24 / 44	12 / 17	40 / 48	41 / 51



(a) Input image with noise. (b) Kernel 3×3. (c) Kernel 5×5. (d) Kernel 7×7.

Fig. 2: Edge detection examples using the Canny Algorithm.

Fig. 3 presents PfCf and power reduction to each kernel size, tracing the trade-off between them to find the most suitable case for the application considering the project constraints. This analysis has stressed that: 1) it is better to explore approximation in multiple adders instead of restricting to a single level, despite the quality benefits due to the slight power reduction reached; 2) approximation C5, which is single level approximation on Level 5, shows an aggressive impact on quality, with a slight improvement on power-efficiency; and 3) the highest power reduction provided by C17 (i.e., all the levels approximated) substantially impacts in PfCf results. Therefore, C16 and C12 present the best trade-offs, where the approximation is applied to levels 1-4 and 1-3 in the architectures, respectively, as it has the largest PfCf and power reduction.

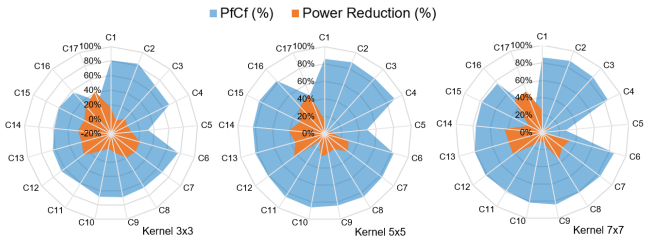


Fig. 3: PfCf and power reduction trade-off to each kernel for all approximation cases evaluated.

IV. Conclusion

In this paper, we evaluated three different kernel sizes to Gaussian filter. Each kernel implementation was divided into five different levels to assess the impact of the approximation technique. Among all the evaluated cases, area and power reduction achieved up to 40% and 48%, respectively. Considering the trade-off between area, power, and PfCf, the best results were obtained by exploring approximate adders in all initial levels, keeping level 5 without approximation, for C12 and C16 cases.

In an overall evaluation, for power-constrained applications, kernel 3×3 is more suitable as it has the lowest power consumption with more than 57% of PfCf. In contrast, for non-power-constrained applications, kernel 5×5 has the best trade-off between PfCf and power, while kernel 7×7 is preferable to applications without power constraints and targeting low-detailed images. The evaluation presented in this work shows space for developing a configurable Gaussian filter architecture that could adapt to the power and quality run-time demands of image processing applications.

References

[1] A. Sampson, *Hardware and Software for Approximate Computing*. Ph.D. in computer science & engineering, UW, Seattle, WA, USA, 2015.

[2] F. Pellegrino, W. Vanzella, and V. Torre, "Edge detection revisited," *IEEE Trans. SMC*, vol. 34, no. 3, pp. 1500–1518, 2004.

[3] J. Canny, "A computational approach to edge detection," *IEEE TPAMI*, vol. PAMI-8, no. 6, pp. 679–698, 1986.

[4] L. B. Soares *et al.*, "Design methodology to explore hybrid approximate adders for energy-efficient image and video processing accelerators," *IEEE TCAS I*, vol. 66, no. 6, pp. 2137–2150, 2019.

[5] J. R. de Oliveira *et al.*, "Exploiting approximate adder circuits for power-efficient Gaussian and Gradient filters for Canny edge detector algorithm," in *LASCAS'16*, pp. 379–382, IEEE, Mar. 2016.

[6] J. Lee, H. Tang, and J. Park, "Energy efficient Canny edge detector for advanced mobile vision applications," *IEEE TCSVT*, vol. 28, no. 4, pp. 1037–1046, 2018.

[7] V. Gupta *et al.*, "Low-power digital signal processing using approximate adders," *IEEE TCAD*, vol. 32, pp. 124–137, Jan. 2013.

[8] A. B. Kahng and S. Kang, "Accuracy-configurable adder for approximate arithmetic designs," in *DAC'12*, pp. 820–825, ACM, 2012.

[9] T. A. Borges, L. B. Soares, and C. Meinhardt, "A fine-grained methodology for accuracy-configurable and energy-efficient Gaussian filters design," in *SBCCI'20*, pp. 1–6, IEEE, Aug. 2020.

[10] M. Monteiro *et al.*, "Design of energy-efficient Gaussian filters by combining refactoring and approximate adders," in *ISCAS'21*, pp. 1–5, IEEE, 2021.

[11] M. Hajabdollahi, S. Samavi, and N. Karimi, "Error compensation and hardware reduction of fixed point 2-D Gaussian filter," in *MVIP'15*, pp. 84–87, IEEE, Nov. 2015.

[12] Y. Voronenko and M. Püschel, "Multiplierless multiple constant multiplication," *ACM Trans. Algorithms*, vol. 3, p. 11, May 2007.

[13] L. H. Lourenço, D. Weingaertner, and E. Todt, "Efficient implementation of Canny edge detection filter for ITK using CUDA," in *WSCAD'12*, pp. 33–40, IEEE, Oct. 2012.

[14] D. Martin *et al.*, "A database of human segmented natural images and its application to evaluating segmentation algorithms and measuring ecological statistics," in *IPCV'01*, vol. 2, pp. 416–423, IEEE, July 2001.

[15] J. Castro-Godínez, S. Esser, M. Shafique, S. Pagani, and J. Henkel, "Compiler-driven error analysis for designing approximate accelerators," in *DATE'18*, pp. 1027–1032, Mar. 2018.